

Part 6: Multi-Tier Agent Architecture

Building Hierarchical Systems with Agentic Coding Tools

Advanced Agentic Coding Tools Course

University of South Carolina

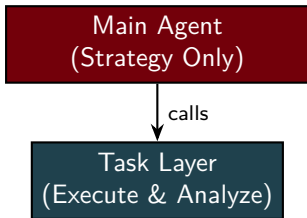
January 23, 2026

Outline

- 1 Beyond Two Layers
- 2 Three-Tier Framework
- 3 Hierarchical Multi-Agent Systems (HMAS)
- 4 Agents Calling Agents
- 5 Five-Phase Workflow
- 6 Best Practices
- 7 Summary

Why Stop at Two Layers?

Two-Layer Limitation:



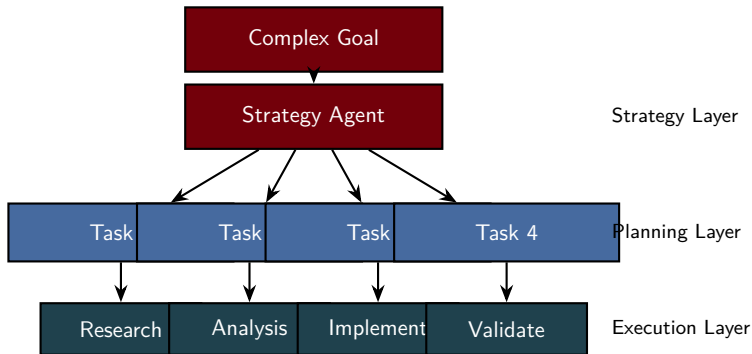
Issues:

- All complexity forced into task execution
- Poor separation of concerns
- Limited nuanced decision handling
- No intermediate planning layer
- Difficult multi-domain management

Key Insight

Real-world problems require multiple levels of abstraction that do not fit neatly into “main agent” and “task.”

Complex Problem Decomposition



Multi-Level Decomposition Benefits:

- **Strategy:** High-level reasoning about what to do
- **Planning:** How to break this into manageable pieces
- **Execution:** Specific actions to accomplish each piece

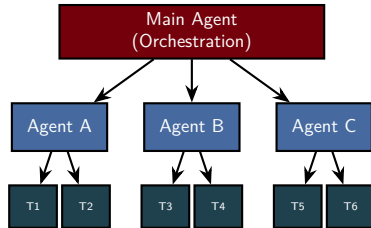
Scaling Through Delegation

Linear (2-layer):

- Size of work → Complexity of main agent
- One agent handles everything
- Problem: Cognitive overload

Hierarchical (3+ layer):

- Size of work → More agents
- Distributed problem solving
- Parallelization & specialization



Parallelization Benefits

Multiple tasks execute simultaneously, better resource utilization, faster completion, specialized expertise at each level

When to Use Multi-Tier Systems

Multi-Tier Systems Excel At:

- ✓ Large codebase modifications
- ✓ Research and analysis tasks
- ✓ Multi-phase projects
- ✓ Cross-domain problems
- ✓ Multiple perspectives needed
- ✓ High availability requirements

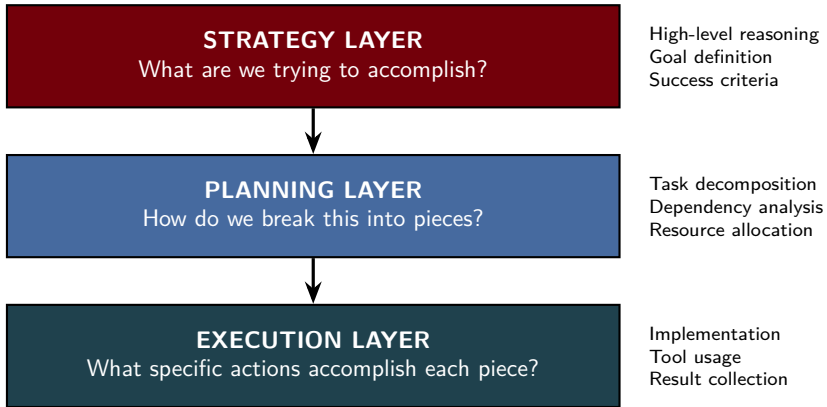
Common Scenarios:

- 1. Software Engineering:**
Architecture → Design → Implementation
- 2. Research:**
Lit review → Analysis → Writing
- 3. Data Processing:**
Validate → Transform → Report
- 4. DevOps:**
Plan → Configure → Deploy

Decision Criteria

Use multi-tier when: problem naturally decomposes into levels, different expertise needed at stages, task size is large, parallelization is beneficial, results need intelligent aggregation.

The Three-Tier Foundation



Why Three Layers?

- Beyond 3: diminishing returns, increased latency

This mirrors:

- Human problem-solving approaches
- OpenHands, AutoGPT architectures

Strategy Layer Deep Dive

Strategy Layer Responsibilities:

- Receive high-level goal
- Analyze problem domain
- Identify constraints
- Define success criteria
- Make major strategic decisions
- Decompose into major phases
- Output: Strategic Plan

Key Characteristics:

- Highest-level abstraction
- Domain-specific reasoning
- Uses external knowledge

Example Input

“Build a data pipeline to process customer transaction logs and generate weekly reports”

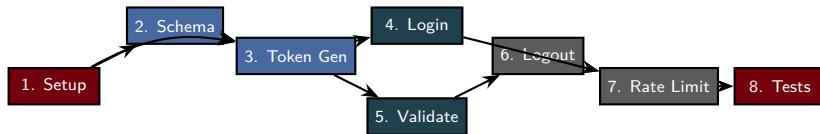
Processing Steps

1. Understand problem scope
2. Identify major phases
3. Determine key technologies
4. Define quality standards
5. Plan resource allocation

Planning Layer Deep Dive

Planning Layer Responsibilities:

- Receive strategic plan
- Decompose into executable tasks
- Analyze dependencies
- Optimize execution order
- Allocate resources

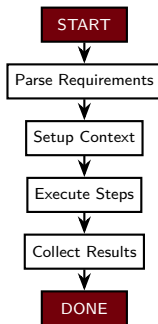


Dependency Graph for OAuth2 Implementation

Planning Strategies: Topological sorting, Parallelization identification, Load balancing, Error recovery planning

Execution Layer Deep Dive

Execution Task Lifecycle:



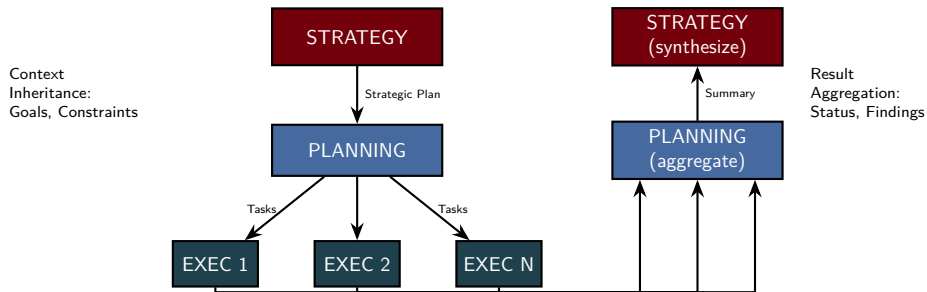
Available Tools:

- File operations (read, write, edit)
- Code execution (run tests, lint)
- Build systems
- Package managers
- Version control
- APIs and external services

Execution Patterns:

- Deterministic when possible
- Retry logic for transient failures
- Progress reporting
- Resource cleanup

Information Flow Between Layers



Downward Flow:

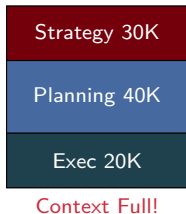
- High-level goals
- Constraints
- Key decisions
- NOT: Implementation details

Upward Flow:

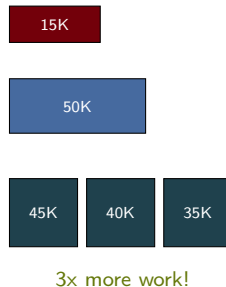
- Completed results
- Status and errors
- Artifacts
- NOT: Trial-and-error logs

Context Window Management

Two-Layer System:



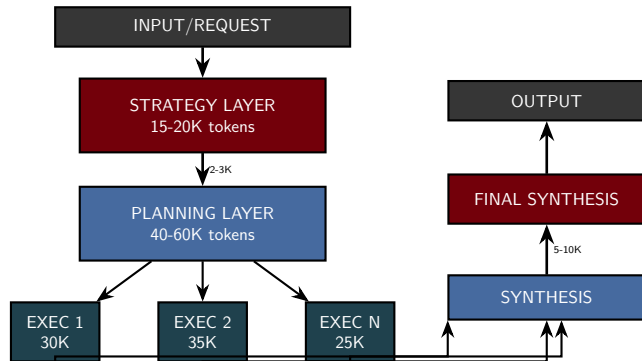
Three-Layer System:



Context Optimization Techniques

- Progressive summarization upward
- Selective context passing downward
- File references instead of full contents

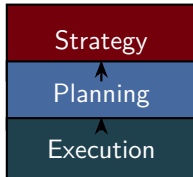
Complete Three-Tier Architecture



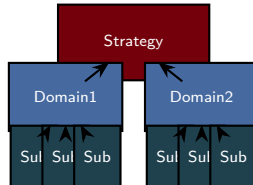
Key Observation: Execution agents run in parallel. Total time = Strategy + Planning + max(Exec times) + Synthesis, NOT sum of all exec times.

Introduction to HMAS

Simple Three-Tier:



Hierarchical Multi-Agent:



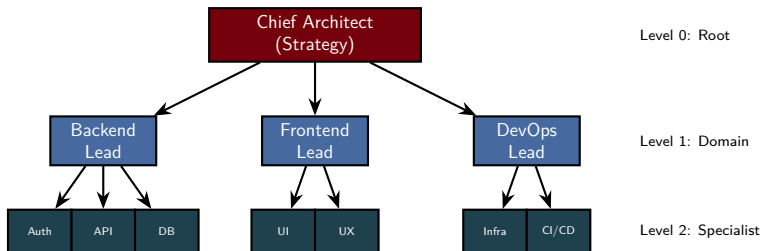
HMAS Characteristics:

- Tree or graph structure of agents
- Each parent agent coordinates children
- Specialization at each level
- Can be arbitrarily deep
- Suitable for very complex problems

HMAS Enables

Specialization at each level. Domain specific agents. Clear reporting structure. Scalable to

Hierarchical Agent Tree Design



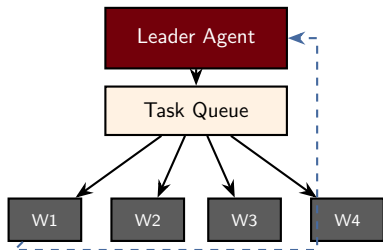
Tree Characteristics:

- Narrow (2-3 children): Deep, specialized
- Wide (5-10 children): Flat, general
- Balanced (3-5): Recommended

Parallelization:

- All siblings run in parallel
- Each level multiplies parallelism
- Total speedup: product of sibling counts

Leader-Worker Pattern



Leader Responsibilities:

1. Task Distribution
2. Monitoring Progress
3. Result Aggregation
4. Error Recovery

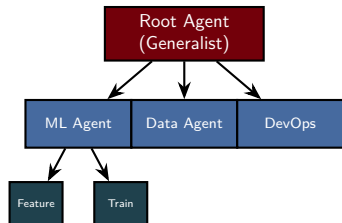
Load Balancing Strategies:

- **Round-Robin:** Simple, fair
- **Work Queue:** FIFO, adaptive
- **Priority Queue:** Optimizes latency
- **Capacity-Based:** Balanced utilization

Worker Pool Management:

- Dynamic sizing (scale with load)
- Fixed sizing (predictable)
- Hybrid (core + temporary)

Domain-Specialized Sub-Agents



Benefits of Specialization:

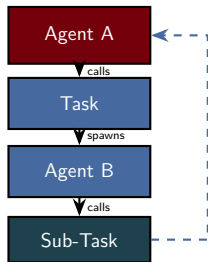
1. **Domain Expertise:** Better decisions
2. **Reduced Context:** Better token efficiency
3. **Reusability:** Library of experts
4. **Parallel Development:** No blocking

Specialization Patterns

- **Domain Specialists:** ML, Dev, Ops (distinct areas)
- **Task-Phase Specialists:** Design, Code, Test (workflow phases)
- **Technology Specialists:** Backend, Frontend, Database (tech stack)
- **Hybrid:** Mix of domain and phase specialists

Hierarchical Delegation Patterns

Fundamental Pattern:



Key Difference:

- Before: Main agent calls tasks (passive)
- Now: Tasks can spawn agents
- Enables: Recursive delegation

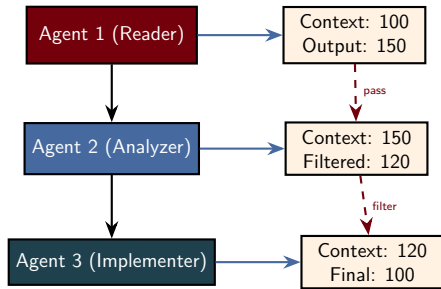
Benefits:

1. Delegation to specialists
2. Specialization at each level
3. Scalability (recursive decomposition)
4. Reusability of agent components

Orchestration Models

Hierarchical Delegation, Delegation with Collaboration, Recursive Decomposition, Dynamic

Context Inheritance Flow



Context Inheritance Strategies:

Chain: Result passes through each step

Filtered: Only relevant context passes

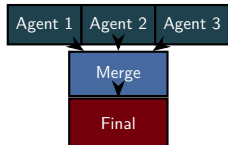
Reference-Based: Pointers instead of content

Result Consolidation Patterns

Sequential Consolidation:



Parallel Consolidation:



Merge Strategies:

- **Concatenate:** $A + B + C$
- **Deduplicate:** Remove duplicates
- **Verify:** Check contradictions
- **Synthesize:** Create unified view

Conflict Resolution:

1. Priority-Based (architecture wins)
2. Vote-Based (majority wins)
3. Consequence-Based (evaluate impact)
4. Escalation (return to main)

Limitations of Nested Agent Calls

Key Limitation: Claude Code Cannot Use Nested Subagents

Tasks cannot spawn SubAgents from within tasks. All agents must be called from the main agent level.

Why This Limitation Exists:

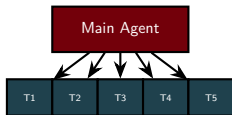
- **Token Budget:** Nested calls multiply usage
- **Context Contamination:** Grows exponentially
- **Latency:** Each level adds overhead
- **Complexity:** Error handling becomes difficult

Theoretical Depth Limits:

Depth	Token Usage	Status
Depth 1	27K tokens	OK
Depth 2	64K tokens	OK

Workaround Patterns

Pattern 1: Flat Delegation



All coordination at main level

Pattern 2: Role-Based Shared State

- Shared context between agents
- Reader → Planner → Coder → Tester

Pattern 3: Hierarchical Tasks

- Subtasks with own context
- Simulates deeper nesting

Pattern 4: Pipeline with Files

- Agent 1 → File 1 → Agent 2
- Coordination through artifacts

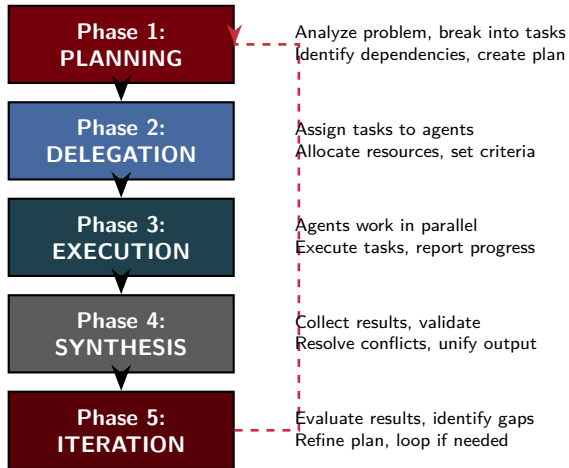
Pattern 5: Domain Specialists

- Different main agents per domain
- Root orchestrator coordinates

Recommendation

Claude Code uses Pattern 1 (Flat Delegation). Most problems can be solved with 3-5 parallel

The Five-Phase Model



Five-Phase Workflow Details

Phase 1: PLANNING

- Parse request
- Understand domain
- Identify major phases
- Analyze dependencies
- Estimate resources

Phase 2: DELEGATION

- Categorize tasks
- Select appropriate agents
- Prepare context for each
- Determine parallelization

Phase 3: EXECUTION

- Spawn agents
- Execute tasks
- Handle local errors
- Collect results

Phase 4: SYNTHESIS

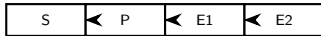
- Check completeness
- Validate consistency
- Integrate outputs
- Generate report

Phase 5: ITERATION

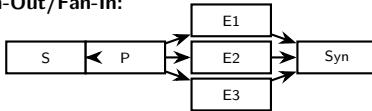
- Evaluate against criteria

Agent Delegation Patterns

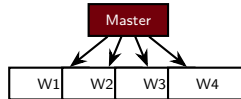
Pipeline:



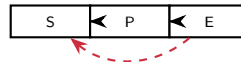
Fan-Out/Fan-In:



Master-Worker:



Feedback Loop:



Pattern Selection Guide

Pipeline: Strict ordering | **Fan-Out:** Parallel then consolidate | **Master-Worker:** Many similar tasks | **Feedback:** Iterative refinement

Best Practices for Multi-Tier Systems

1. Clear Layer Boundaries

- ✓ Each layer has specific responsibilities
- ✓ Information flows through defined interfaces
- ✗ Strategy layer executing code

2. Explicit Success Criteria

- ✓ Define how layer knows when done
- ✓ Each layer validates its work
- ✗ Ambiguous success definitions

3. Context Management

- ✓ Sufficient, not excessive, context
- ✓ Summarize results upward

4. Error Handling

- ✓ Explicit error reporting with root cause
- ✓ Graceful degradation and fallbacks
- ✗ Silent failures

5. Monitoring and Logging

- ✓ Track execution through all layers
- ✓ Log decision points and reasoning
- ✗ Ignore performance bottlenecks

6. Testing

- ✓ Unit test each layer independently
- ✓ Integration test layer interactions
- ✗ Only test individual agents

Failure Modes and Recovery

STRATEGY FAILURE

Impact: All work wasted
Recovery: Restart strategy

PLANNING FAILURE

Impact: Agents fail/deadlock
Recovery: Re-plan

EXECUTION FAILURE

Impact: May block deps
Recovery: Retry task

Recovery: Retry → Escalate → Fallback → Manual

Recovery Strategies:

- **Simple Fallback:** Use prior version
- **Degraded Mode:** Partial solution OK
- **Manual:** Human intervention
- **Skip:** Continue without task
- **Compensate:** Alternative approach
- **Circuit Breaker:** Stop retrying

Performance Metrics

Speed Metrics:

- Total execution time
- Layer breakdown
- Parallelization ratio
- Coordination overhead

Efficiency Metrics:

- Context efficiency (tokens used/available)
- Task success rate
- Retry rate
- Resource utilization

Quality Metrics:

- Success criteria met (%)
- Error rate
- Result correctness
- User satisfaction

Scalability Metrics:

- Throughput (tasks/time)
- Latency vs. problem size
- Resource growth
- Agent utilization

Speedup Example

Sequential (2 time) = 1 + 2 + 10 + 12 = 25

Key Takeaways

Multi-Tier Architecture:

1. Three-tier is the practical foundation
2. Strategy → Planning → Execution
3. Parallelization at execution layer
4. Context management is critical

HMAS Principles:

1. Tree structure with specialists
2. Leader-worker for parallel work
3. Domain specialization benefits
4. Clear coordination protocols

Five-Phase Workflow:

1. Plan → Delegate → Execute
2. Synthesize → Iterate
3. Feedback loops for refinement

Practical Considerations:

1. Claude Code: flat multi-agent (3-5)
2. Token budgets constrain depth
3. Workaround patterns exist
4. Design for your constraints

Core Insight

Part 6: Multi-Tier Agent Architecture

Building Hierarchical Systems with Agentic Coding Tools

